

Generates code and adversarial attacks from the **same specification**, at the same time — before a commit exists.

Builder and Breaker run concurrently on one spec. Arbiter scores every attack individually. A gate blocks the CI pipeline when the score falls below threshold. No manual test authoring, no separate security review step.

⚡ How CombatPair Works — Live Pipeline

Each run executes this pipeline. Builder and Breaker run **concurrently** — no waiting for one before the other. Arbiter scores each attack. ARS gate decides pass or block. The whole cycle completes in seconds.

**~4–5 min**

median run time, quick mode

ARS < 0.80

default hard gate threshold, configurable

SHA-256

tamper-evident report, verifiable on demand

22 commands

full CLI: setup, run, status, audit, serve

WHERE THIS SITS RELATIVE TO EXISTING TOOLING

SAST, DAST, and penetration testing test code after it exists — after a PR opens, after deployment, after months in production. CombatPair runs at the generation event itself, before any of those tools have anchored to an implementation. It does not replace them; it covers the window before they start.

Coverage across languages, compliance domains, and coding environments

LANGUAGES — AUTO-DETECTED

5 languages, each with its own CWE priority list

Detected directly from the spec — no per-project configuration.

Example: **prototype pollution** (CWE-1321) prioritized for JavaScript; **nil-pointer dereference** (CWE-476) for Go — language-appropriate, not generic.

[Python](#) [JavaScript](#) [TypeScript](#) [Java](#) [Go](#)

COMPLIANCE DOMAINS — BUILT IN

5 regulated-industry playbooks today

Additional domains added via a single YAML file, same schema.

Each playbook maps to the source regulation directly — e.g. FINRA Rules 4370/3110, HIPAA Security Rule (45 CFR 160/164), PCI DSS v4.0.

[FINRA](#) [HIPAA](#) [OWASP Top 10](#) [PCI DSS](#) [SOC 2](#)

MCP SERVER — NATIVE

Connects directly inside the coding tool

One command wires it in — `combatpair integrate`. No separate workflow to context-switch into.

Exposes five tools over the Model Context Protocol: run, status, vault stats, policy list, and integrity verify.

[Claude Code](#) [Cursor](#) [Windsurf](#) [GitHub Copilot](#) [Codex](#)

CI / CD — PIPELINE-NATIVE

Wired into the pipeline you already run

SARIF for code scanning, JUnit XML for test reporting — every attack becomes a test case.

SARIF output uploads directly to GitHub Code Scanning; JUnit XML is read by any CI dashboard already parsing test results.

[GitHub Actions](#) [Jenkins](#) [GitLab CI](#) [CircleCI](#)

CLI — COMPLETE SURFACE

22 commands, setup through audit

Sync (—pretty) or async (—background) execution, same command family.

Covers the full lifecycle: configuration, execution, CI integration, compliance audit, and report integrity verification.

[setup](#) [run](#) [status](#) [findings](#) [audit](#) [verify](#) [serve](#) [vault](#)

DASHBOARD — CENTRALIZED

Every run tracked in one place, in real time

Foreground and background sessions, side by side, auto-refreshing — no polling required.

Resilience-score trend, attack-outcome breakdown, and gate status per run, across every repository connected to the instance.

[HTML export](#) [SARIF export](#) [JUnit export](#)